

Write a program to implement a Web API using ASP.NET that supports the HTTP GET, POST, PUT, and DELETE methods for managing Employee records. Each Employee should contain the following fields: F_Name, L_Name, Email, and Department.

[Model.cs:](#)

```
namespace EmployeeApi.Models
{
    public class Employee
    {
        public int Id { get; set; }

        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string Department { get; set; } = string.Empty;
    }
}
```

[AppDbContext.cs:](#)

```
using Microsoft.EntityFrameworkCore;
using EmployeeApi.Models;
namespace EmployeeApi.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

        public DbSet<Employee> Employees => Set<Employee>();
    }
}
```

[Program.cs:](#)

```
using EmployeeApi.Data;
using EmployeeApi.Models;
using Microsoft.EntityFrameworkCore;
```

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.EnsureCreated();
}

app.UseDefaultFiles();
app.UseStaticFiles();
app.UseHttpsRedirection();

app.MapGet("/api/employees", async (AppDbContext db) =>
    await db.Employees.ToListAsync());

app.MapGet("/api/employees/{id:int}", async (int id, AppDbContext db) =>
    await db.Employees.FindAsync(id) is Employee emp
        ? Results.Ok(emp)
        : Results.NotFound());

app.MapPost("/api/employees", async (Employee employee, AppDbContext db) =>
{
    db.Employees.Add(employee);
    await db.SaveChangesAsync();
    return Results.Created($"/api/employees/{employee.Id}", employee);
});

app.MapPut("/api/employees/{id:int}", async (int id, Employee updated, AppDbContext db) =>
{
    var employee = await db.Employees.FindAsync(id);
    if (employee is null) return Results.NotFound();
    employee.FirstName = updated.FirstName;
    employee.LastName = updated.LastName;
    employee.Email = updated.Email;

```

```
employee.Department = updated.Department;

await db.SaveChangesAsync();

return Results.NoContent();

});

app.MapDelete("/api/employees/{id:int}", async (int id, AppDbContext db) =>
{
    var employee = await db.Employees.FindAsync(id);
    if (employee is null) return Results.NotFound();

    db.Employees.Remove(employee);
    await db.SaveChangesAsync();
    return Results.NoContent();
});

app.Run();
```

OUTPUT:

Page:

  http://localhost:5155/ 

Employee Registry

First Name	Last Name
Email	Department

first last
csit — mail@test.com

Post:

Employee Registry

First Name	Last Name
Email	Department
Add Employee	

first last

Csit — mail@test.com

Keshav Neupane

Csit — keshav@mail.com

Del:

http://localhost:5155/



Employee Registry

First Name	Last Name
Email	Department
Add Employee	

Keshav Neupane

Csit — keshav@mail.com

Put:

Employee Registry

First Name	Last Name
Email	Department
Add Employee	

Keshav

Neupane

keshav@mail.com

Csit

Employee Registry

First Name	Last Name
Email	Department
Add Employee	

Keshav Neupane
CSIT — keshav@mail.com

Edit

Delete

Develop a Web API using ADO.NET for a FACULTY(CourseID, Course_Name, No_of_Semester, Fee) table. Insert some sample faculty records, retrieve all faculty records, and retrieve the Fee of the records having Course_Name as “CSIT”. Use appropriate model classes and ADO.NET for performing the required database operations.

[Model.cs](#)

```
namespace IostFacultyApi.Models
{
    public class Faculty
    {
        public string CourseID { get; set; } = string.Empty;
        public string Course_Name { get; set; } = string.Empty;
        public int No_of_Semester { get; set; }
        public decimal Fee { get; set; }
    }
}
```

[DbInitializer.cs](#)

```
using Microsoft.Data.Sqlite;

namespace IostFacultyApi.Data
{
    public static class DbInitializer
    {
        private const string ConnectionString = "Data Source=IOST.db";

        public static void Initialize()
        {
            using var conn = new SqliteConnection(ConnectionString);
            conn.Open();

            var tableCmd = conn.CreateCommand();
            tableCmd.CommandText = @"
CREATE TABLE IF NOT EXISTS FACULTY (
```

```

        CourseID TEXT PRIMARY KEY,
        Course_Name TEXT NOT NULL,
        No_of_Semester INTEGER NOT NULL,
        fee REAL NOT NULL
    );";

    tableCmd.ExecuteNonQuery();
}
}
}

```

[Program.cs](#)

```

using IostFacultyApi.Data;
using IostFacultyApi.Models;
using Microsoft.Data.Sqlite;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
DbInitializer.Initialize();
app.UseDefaultFiles();
app.UseStaticFiles();
app.UseHttpsRedirection();
const string ConnectionString = "Data Source=IOST.db";

app.MapGet("/api/faculty", () =>
{
    var list = new List<Faculty>();
    using var conn = new SqliteConnection(ConnectionString);
    conn.Open();
    using var cmd = conn.CreateCommand();
    cmd.CommandText = "SELECT CourseID, Course_Name, No_of_Semester, fee FROM FACULTY;";
    using var reader = cmd.ExecuteReader();

```

```

while (reader.Read())
{
    list.Add(new Faculty
    {
        CourseID = reader.GetString(0),
        Course_Name = reader.GetString(1),
        No_of_Semester = reader.GetInt32(2),
        Fee = Convert.ToDecimal(reader.GetDouble(3))
    });
}
return Results.Ok(list);
});

```

```

app.MapGet("/api/faculty/fee/{courseId}", (string courseId) =>
{
    using var conn = new SqlConnection(ConnectionString);
    conn.Open();
    using var cmd = conn.CreateCommand();
    cmd.CommandText = "SELECT CourseID, Course_Name, fee FROM FACULTY WHERE CourseID = @id;";
    cmd.Parameters.AddWithValue("@id", courseId);
    using var reader = cmd.ExecuteReader();
    if (reader.Read())
    {
        var result = new
        {
            CourseID = reader.GetString(0),
            Course_Name = reader.GetString(1),
            Fee = Convert.ToDecimal(reader.GetDouble(2))
        };
        return Results.Ok(result);
    }
});

```



```

    }

    return Results.NotFound(new { message = $"No course found with ID '{courseId}'." });
});

```

```

app.MapPost("/api/faculty", (Faculty faculty) =>
{
    using var conn = new SqlConnection(ConnectionString);
    conn.Open();
    using var cmd = conn.CreateCommand();
    cmd.CommandText = @"
        INSERT INTO FACULTY (CourseID, Course_Name, No_of_Semester, fee)
        VALUES (@id, @name, @semesters, @fee);";
    cmd.Parameters.AddWithValue("@id", faculty.CourseID);
    cmd.Parameters.AddWithValue("@name", faculty.Course_Name);
    cmd.Parameters.AddWithValue("@semesters", faculty.No_of_Semester);
    cmd.Parameters.AddWithValue("@fee", faculty.Fee);

    try
    {
        cmd.ExecuteNonQuery();

        return Results.Created($"/api/faculty/{faculty.CourseID}", faculty);
    }

    catch (SqliteException)
    {
        return Results.BadRequest("CourseID already exists or invalid data.");
    }
});

app.Run();

```

Output:

Default View:

[→](#) [↺](#) <http://localhost:5146/>

IOST Faculty Registry

Add New Course Record

Course ID (e.g. CS101)	Course Name (e.g. CSIT)
No. of Semesters	Fee Amount
Insert Record	

Lookup Fee by Course ID

Enter Course ID	Query Fee
--	--

All Faculty Records

Csit (CS101) Semesters: 3 Fee: \$2000

After Insertion:

All Faculty Records

Csit (CS101)
Semesters: 3 | Fee: \$2000

OS (CS102)
Semesters: 3 | Fee: \$3000

FeeLookup:

Lookup Fee by Course ID

CS101	Query Fee
------------------------------------	--

Course: Csit (CS101)
Fee: \$2000

Develop a Web Application for a Bag Registry using ASP.NET Core Minimal API and Entity Framework Core for a Bag model with properties Brand, Price, and Date. Implement functionality to display all records, insert new bags, update prices by manufacturing year, delete records by year, and query brands priced above a minimum threshold.

[Model.cs](#)

```
namespace BagRegistry.Models
{
    public class Bag
    {
        public int Id { get; set; }
        public string Brand { get; set; } = string.Empty;
        public decimal Price { get; set; }
        public DateTime Date { get; set; }
    }
}
```

[AppDbContext.cs](#)

```
using Microsoft.EntityFrameworkCore;
using BagRegistry.Models;
namespace BagRegistry.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

        public DbSet<Bag> Bags => Set<Bag>();
    }
}
```

[Program.cs](#)

```
using BagRegistry.Data;
using BagRegistry.Models;
```

```

using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite("Data Source=bagstore.db"));
var app = builder.Build()

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.EnsureCreated();
    if (!db.Bags.Any())
    {
        db.Bags.AddRange(
            new Bag { Brand = "Gucci", Price = 1500, Date = new DateTime(2022, 5, 10) },
            new Bag { Brand = "Prada", Price = 450, Date = new DateTime(2022, 8, 15) },
            new Bag { Brand = "Vintage Leather", Price = 800, Date = new DateTime(1981, 3, 22) },
            new Bag { Brand = "Samsonite", Price = 300, Date = new DateTime(2020, 1, 1) }
        );
        db.SaveChanges();
    }
}

app.UseDefaultFiles();
app.UseStaticFiles();
app.UseHttpsRedirection();

app.MapGet("/api/bags", async (AppDbContext db) =>
    await db.Bags.ToListAsync());

app.MapPost("/api/bags", async (Bag bag, AppDbContext db) =>

```

```

{
    db.Bags.Add(bag);
    await db.SaveChangesAsync();
    return Results.Created($"/api/bags/{bag.Id}", bag);
});

app.MapPut("/api/bags/update-price", async (int year, decimal newPrice, AppDbContext db) =>
{
    var bagsToUpdate = await db.Bags.Where(b => b.Date.Year == year).ToListAsync();
    if (!bagsToUpdate.Any()) return Results.NotFound(new { message = $"No bags found for year {year}." });

    foreach (var bag in bagsToUpdate)
    {
        bag.Price = newPrice;
    }

    await db.SaveChangesAsync();

    return Results.Ok(new { message = $"Updated {bagsToUpdate.Count} record(s) from year {year} to price {newPrice}." });
});

app.MapDelete("/api/bags/delete-by-year", async (int year, AppDbContext db) =>
{
    var bagsToDelete = await db.Bags.Where(b => b.Date.Year == year).ToListAsync();
    if (!bagsToDelete.Any()) return Results.NotFound(new { message = $"No bags found for year {year}." });
    db.Bags.RemoveRange(bagsToDelete);
    await db.SaveChangesAsync();

    return Results.Ok(new { message = $"Deleted {bagsToDelete.Count} record(s) from year {year}." });
});

app.MapGet("/api/bags/brands-above-price", async (decimal minPrice, AppDbContext db) =>
{
    var brands = await db.Bags

```

```

        .Where(b => b.Price > minPrice)

        .Select(b => new { b.Brand, b.Price })

        .ToListAsync();

    return Results.Ok(brands);

});

app.Run();

```

OUTPUT:

PageOverView:

The screenshot shows a web browser window with the address `http://localhost:5078/`. The page title is "BAG Registry Management". The page contains several form sections:

- Add Bag:** A form with three input fields: "Brand", "Price", and a date field (highlighted with a yellow border) containing "mm/dd/yyyy". Below these fields is an "Add" button.
- 1. Update Price by Year:** A form with two input fields: "Year" and "New Price", followed by an "Update" button.
- 2. Delete by Year:** A form with one input field: "Year", followed by a "Delete" button.
- 3. Search Brands Above Price:** A form with one input field: "Min Price", followed by a "Search" button.
- Bag Records:** A table displaying three records:

#1 Gucci - NRS. 2000 (2022-05-10)
#2 Prada - NRS. 2000 (2022-08-15)
#3 Vintage Leather - NRS. 800 (1981-03-22)

Bag Insert:

Bag Records

#1 Gucci - NRS. 2000 (2022-05-10)
#2 Prada - NRS. 2000 (2022-08-15)
#3 Vintage Leather - NRS. 800 (1981-03-22)
#5 King - NRS. 3000 (2026-08-02)

Update price by date:

1. Update Price by Year

Updated 1 record(s) from year 2026 to price \$4000.

#5 King - NRS. 4000 (2026-08-02)

Delete by year:

2. Delete by Year

Deleted 1 record(s) from year 2026.

3. Search Brands Above Price

Bag Records

#1 Gucci - NRS. 2000 (2022-05-10)

#2 Prada - NRS. 2000 (2022-08-15)

#3 Vintage Leather - NRS. 800 (1981-03-22)

Search Above:

3. Search Brands Above Price

Gucci (\$2000), Prada (\$2000), Vintage Leather (\$800)

Design an HTML user registration form with fields for User Name, Password, and Phone Number. Use jQuery to validate the form by ensuring that the User Name and Password are not empty and that the Phone Number contains exactly 10 digits. Display appropriate error messages for invalid inputs and a success message when all validation requirements are satisfied.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>User Registration</title>

  <style>

    body { font-family: system-ui, sans-serif; max-width: 360px; margin: 2rem auto; padding: 0 1rem; }

    .group { margin-bottom: 0.8rem; }

    label { display: block; font-weight: 600; margin-bottom: 0.2rem; }

    input, button { width: 100%; padding: 0.5rem; font-size: 1rem; box-sizing: border-box; }

    button { background: #0066cc; color: white; border: none; border-radius: 4px; cursor: pointer; }

    button:hover { background: #004999; }

    .error { color: #d9534f; font-size: 0.85rem; margin-top: 0.2rem; display: block; }

    .success { color: #28a745; font-weight: bold; margin-top: 1rem; text-align: center; }

  </style>

  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

</head>

<body>

  <h2>User Registration</h2>

  <form id="userForm">

    <div class="group">

      <label for="username">User Name</label>

      <input type="text" id="username" placeholder="Enter username">

      <span class="error" id="usernameError"></span>

    </div>
```



```

<div class="group">
  <label for="password">Password</label>
  <input type="password" id="password" placeholder="Enter password">
  <span class="error" id="passwordError"></span>
</div>

<div class="group">
  <label for="phone">Phone Number</label>
  <input type="text" id="phone" placeholder="10-digit number">
  <span class="error" id="phoneError"></span>
</div>

<button type="submit">Submit</button>
</form>

<div id="formSuccess" class="success"></div>

<script>
  $('#userForm').on('submit', function (e) {
    e.preventDefault();

    $('#error, #formSuccess').text("");

    const user = $('#username').val().trim();
    const pass = $('#password').val().trim();
    const phone = $('#phone').val().trim();

    let valid = true;

    if (!user) { $('#usernameError').text('User name is required. '); valid = false; }
    if (!pass) { $('#passwordError').text('Password is required. '); valid = false; }
    if (!phone) {
      $('#phoneError').text('Phone number is required. ');
      valid = false;
    } else if (!/^d{10}$/.test(phone)) {
      $('#phoneError').text('Must be exactly 10 digits. ');
      valid = false;
    }

    if (valid) {

```

```

        $('#formSuccess').text('Form submitted successfully!');

        this.reset();

    }

});

</script>

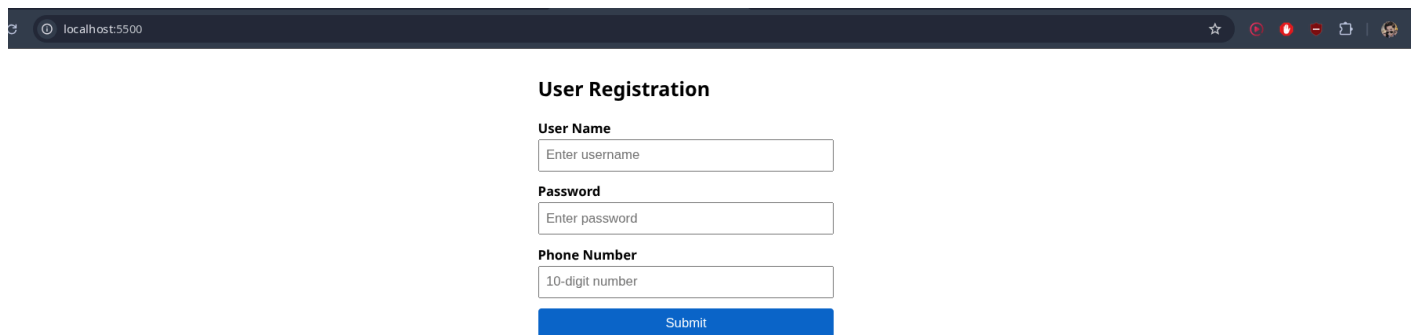
</body>

</html>

```

Output:

Overview:



The screenshot shows a web browser window with the address bar displaying 'localhost:5500'. The page content is a 'User Registration' form. It includes three input fields: 'User Name' with placeholder 'Enter username', 'Password' with placeholder 'Enter password', and 'Phone Number' with placeholder '10-digit number'. A blue 'Submit' button is located below the input fields.

All Error:

User Registration

User Name

User name is required.

Password

Password is required.

Phone Number

Phone number is required.

Submit

Success:

User Registration

User Name

Password

Phone Number

Submit

Form submitted successfully!

Implement an ASP.NET Core Minimal App that implements session management with endpoints to set user session data on login, retrieve current session details, and clear session data on logout.

[Controller.cs](#)

```
using Microsoft.AspNetCore.Mvc;
namespace WebApp.Controllers;
public record UserLogin(string Username, string Role);
public class SessionController : Controller {
    [HttpGet]
    public IActionResult Login() {
        return View();
    }
    [HttpPost]
    public IActionResult Login([FromForm] UserLogin login) {
        if (string.IsNullOrEmpty(login.Username)) {
            ModelState.AddModelError("", "Username cannot be empty.");
            return View(login);
        }
        HttpContext.Session.SetString("Username", login.Username);
        HttpContext.Session.SetString("Role", string.IsNullOrEmpty(login.Role) ? "User" : login.Role);
        HttpContext.Session.SetString("LoginTime", DateTime.Now.ToString("g"));
        return RedirectToAction(nameof(UserProfile));
    }
    [HttpGet]
    public IActionResult UserProfile() {
        var username = HttpContext.Session.GetString("Username");

        if (string.IsNullOrEmpty(username)) {
            return RedirectToAction(nameof(Login));
        }
    }
}
```

```

        ViewBag.Username = username;

        ViewBag.Role = HttpContext.Session.GetString("Role");

        ViewBag.LoginTime = HttpContext.Session.GetString("LoginTime");

        ViewBag.SessionId = HttpContext.Session.Id;

        return View();
    }

    [HttpPost]
    public IActionResult Logout() {
        HttpContext.Session.Clear();

        return RedirectToAction(nameof(Login));
    }
}

```

[Program.cs](#)

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDistributedMemoryCache();

builder.Services.AddSession(options => {
    options.IdleTimeout = TimeSpan.FromMinutes(20);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

var app = builder.Build();

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseSession();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Session}/{action=Index}/{id?}");

app.Run();

```

Output:

Page OverView:

Session Management Demo

Start Session (Login)

Set Session Data

Active Session Status

No active session found.

Session Create:

Active Session Status

Active User: Keshav

Role: Admin

Login Time: 8/23/2026 12:35 PM

Session ID: e0d3c734-5c1b-ea2d-1dfb-6c66b50ccc33

Destroy Session (Logout)

Session Destroy:

Active Session Status

No active session found.

Implement cookie management in ASP.NET Core using Minimal API endpoints to create, read, and delete HTTP cookies, supported by an HTML interface for user interaction.

[Controller.cs](#)

```
using Microsoft.AspNetCore.Mvc;
```

```
namespace CookieManagementApp.Controllers {  
    [ApiController]  
    [Route("api/[controller]")]  
    public class CookiesController : ControllerBase {  
  
        [HttpPost("set")]  
        public IActionResult SetCookie([FromBody] CookieRequest req) {  
            if (string.IsNullOrEmpty(req.Name)) {  
                return BadRequest("Cookie name is required.");  
            }  
  
            var options = new CookieOptions {  
                Expires = DateTimeOffset.UtcNow.AddDays(req.Days),  
                HttpOnly = false,  
                SameSite = SameSiteMode.Lax,  
                Path = "/"  
            };  
  
            Response.Cookies.Append(req.Name, req.Value ?? string.Empty, options);  
            return Ok(new { message = $"Cookie '{req.Name}' set successfully." });  
        }  
  
        [HttpGet("get/{name}")]  
        public IActionResult GetCookie(string name) {  
            if (Request.Cookies.TryGetValue(name, out var value)) {  
                return Ok(new { name, value });  
            }  
        }  
    }  
}
```

```

    }

    return NotFound(new { message = "Cookie not found." });
}

```

```

[HttpDelete("delete/{name}")]
public IActionResult DeleteCookie(string name) {
    Response.Cookies.Delete(name);
    return Ok(new { message = $"Cookie '{name}' deleted." });
}

```

```

[HttpGet("all")]
public IActionResult GetAllCookies() {
    var cookies = Request.Cookies.ToDictionary(c => c.Key, c => c.Value);
    return Ok(cookies);
}
}

```

```

public record CookieRequest(string Name, string Value, int Days);
}

```

[Program.cs](#)

```

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddRazorPages();
builder.Services.AddControllers();
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.MapRazorPages();
app.MapControllers();
app.Run();

```

Output:

Page Overview:

 http://localhost:5120/index.html  

JavaScript Cookie Management

Set / Update Cookie

Cookie Name (e.g. user_theme)

Cookie Value (e.g. dark_mode)

Expiry in Days (e.g. 7)

Save Cookie

Read or Delete Specific Cookie

Enter Cookie Name

Read

Delete

All Active Cookies (`document.cookie``)

No active cookies

Cookie Addition:

JavaScript Cookie Management

Set / Update Cookie

username

Keshav

10

Save Cookie

All Active Cookies (`document.cookie`)

username=Keshav

Cookie Read:

Read or Delete Specific Cookie

ReadDelete

username: Keshav

Cookie Delete:

Read or Delete Specific Cookie

ReadDelete

Cookie 'username' deleted.

All Active Cookies (`document.cookie`)

No active cookies